Tugas Analisis Proses pada Sistem E-Commerce

Nama: Fransiskus Xaverius Kevin
NIM: 282403656

A. Process Description

1. Identifikasi 5 Proses

Proses adalah program in execution, yaitu entitas yang dapat dijadwalkan dan dieksekusi oleh prosesor.

PID	Program / Kode	Current State	Resource Dipakai	Resource Ditunggu
P1	Payment Service: process_payment()	Ready (sudah dapat respons database)	Konteks CPU tersimpan, memori, koneksi database	CPU / dispatcher
P2	Order Service: insert_order()	Blocked / Waiting (interrupt I/O, context switching)	Memori, buffer database	Penyelesaian I/O dan CPU
P3	Email Service: send_confirmation()	Blocked (order belum tercatat)	Memori	Data order dari P2 dan CPU
P4	Database Backup: backup_db()	Blocked / Waiting (menunggu disk)	Memori	Disk (dipakai proses lain)
P5	Fraud Detection: analyze_transaction()	Ready (quantum habis, didahului P2)	Konteks CPU tersimpan	CPU

Catatan P2: jika interrupt bersifat eksternal murni, statusnya dapat dimodelkan sebagai Ready (preempted). Di sini dimodelkan sebagai Blocked / Waiting karena eksekusinya terhenti oleh interrupt I/O dan terjadi context switching.

2. Process Control Block (PCB) untuk P1

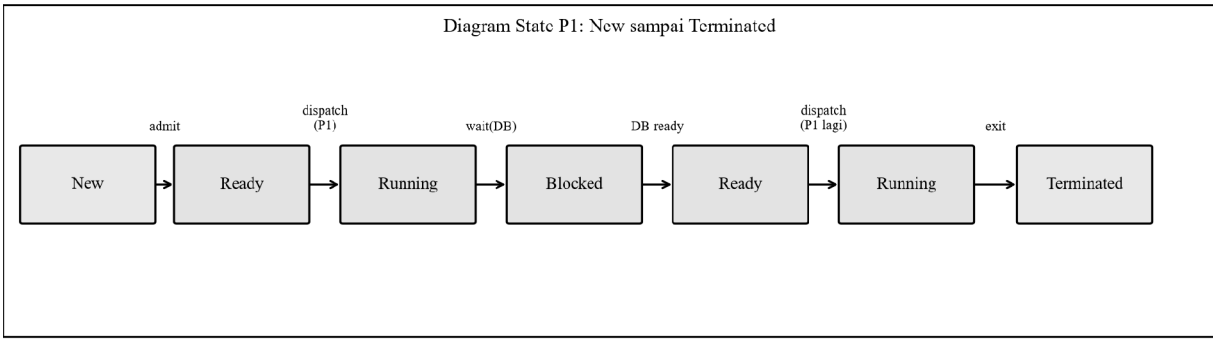
PCB adalah struktur data OS yang menyimpan seluruh informasi proses agar proses dapat dihentikan dan dilanjutkan kembali. Acuan: Tanenbaum, Figure 2.4.

Field	Isi untuk P1
Process ID	1
Process state	Ready (sebelumnya Blocked)
Program counter	Alamat lanjutan setelah wait(DB_response)
Registers, Stack pointer, PSW	Nilai tersimpan saat P1 diblokir
Priority / scheduling	Prioritas, sisa quantum, CPU time used

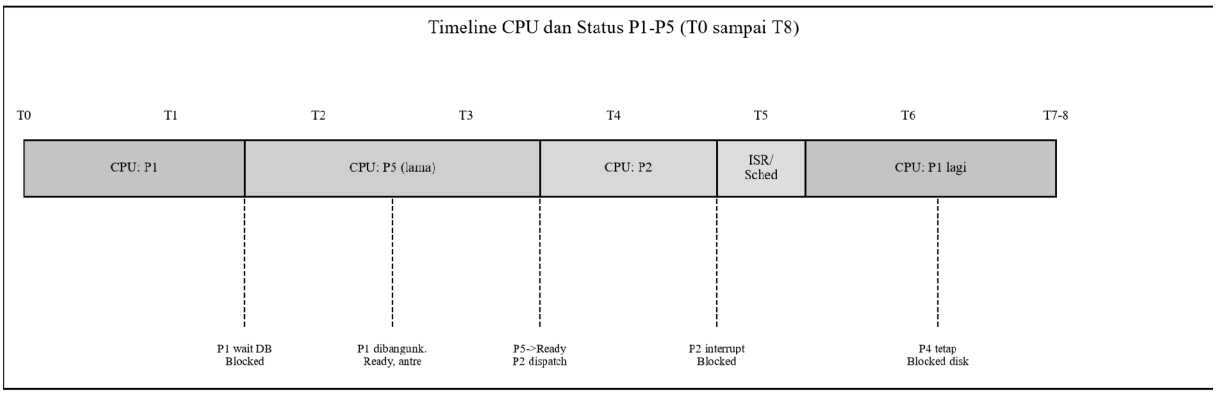
Memory management	Pointer text, data, stack segment
File management	File descriptors, User ID, Group ID
Accounting	Waktu mulai, CPU time, waiting time
Wait info	DB_response selesai, antre dispatcher

B. Process State P1 (Detail dengan P1-P5)

Model yang dipakai: New → Ready → Running → Blocked / Waiting → Ready → Running → Terminated.



Gambar 1. Alur state P1 dari New sampai Terminated.



Gambar 2. Timeline CPU dan status P1-P5 (T0 sampai T8).

Waktu	**CPU**	**P1**	**Lainnya**	**Transisi**
T0	idle → P1	New → Ready → Running	P2 Ready, P3 Blocked, P4 Blocked, P5 Ready	admit, dispatch
T1	P1	Running	P2, P5 Ready; P4 Blocked	proses transaksi
T2	P1 → P5	Running → Blocked	P5 Ready → Running	wait DB, switch
T3	P5	Blocked	P5 lama; P2 Ready	quantum panjang
T4	P5	Blocked → Ready	P5 Running; P1 antre	DB interrupt

T5	P5 → P2	Ready antre	P5 → Ready; P2 → Running	pilih P2
T6	P2 → sched	Ready antre	P2 → Blocked; P4 Blocked	I/O interrupt
T7-8	→ P1	Ready → Running → Terminated	P2 Blocked, P5 Ready, P4 Blocked	dispatch, exit

1. New menjadi Ready: P1 dibuat, masuk Ready queue bersama P2 dan P5. P3 Blocked menunggu data order, P4 Blocked menunggu disk.
2. Ready menjadi Running: dispatcher memilih P1, me-restore PC, register, PSW, dan stack-pointer dari PCB P1.
3. Running menjadi Blocked: P1 meminta database dan menunggu. OS menyimpan konteks ke PCB, P1 masuk wait queue, CPU dibebaskan.
4. Switch ke P5: scheduler memilih P5, restore PCB P5. P5 Running lama karena CPU-bound.
5. Blocked menjadi Ready: respons database tiba lewat interrupt. P1 masuk Ready queue, tidak langsung preempt karena P5 masih Running.
6. P5 menjadi Ready dan P2 menjadi Running: quantum habis, scheduler memilih P2. P1 tetap antre.
7. P2 menjadi Blocked: interrupt I/O datang, OS simpan PCB P2 dan context switching. P4 tetap Blocked karena disk busy.
8. Ready menjadi Running dan Terminated: P1 dapat giliran, selesaikan pembayaran, normal exit. Suspend tidak dipakai.

C. Process Control

1. Apakah terjadi context switching saat P1 menunggu database dan CPU diberikan ke P5? Ya. Ketika P1 tidak dapat melanjutkan karena menunggu database, OS masuk kernel mode, menyimpan program counter, register, stack pointer, dan PSW P1 ke PCB, mengubah state P1 menjadi Blocked dan memasukkannya ke wait queue database. Kemudian scheduler memilih P5 dan me-restore PCB P5 ke CPU. Inilah context switching. Mekanisme ini adalah inti multiprogramming sehingga utilisasi CPU adalah $1 - p^n$. Pergantian hanya dilakukan saat blokir, quantum habis, atau interrupt karena ada biaya overhead.

2. Apakah P1 otomatis langsung mendapat CPU setelah kembali Ready? Tidak. Ready berarti layak jalan, bukan sedang jalan. CPU berikutnya ditentukan scheduler dan dispatcher dari Ready queue (FIFO, prioritas, round robin; preemptive vs non-preemptive). Buktinya setelah P1 Ready justru P2 yang mendapat CPU, lalu P2 kena interrupt I/O. Pada

non-preemptive P1 menunggu Running selesai; pada preemptive P1 hanya mendahului jika prioritasnya lebih tinggi.